

Programmierung

Vorlesung, Übung und Tutorium

06 – Datenstrukturen

Prof. Dr. Andreas Biesdorf

Tutorium: Nicolas Pfeifer

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Agenda

1 Software Engineering vs.
Programmierung

2 Testen in Python

3 Argumente und
Kommandozeilenbefehle

4 Datei-Handling und IO

5 Algorithmen und Datenstrukturen

6 Such- und Sortierverfahren



Portfolioprüfung

- **Algorithmen** werden eingesetzt, um **dynamische Mengen von Elementen** zu **manipulieren**, wobei dies verschiedene Operationen umfassen kann, wie zum Beispiel das **Suchen, Einfügen, Löschen** oder **Finden** des Minimums bzw. Maximums, oder das Identifizieren von Elementen, die kleiner oder größer als ein gegebenes Element sind.
- **Datenstrukturen** spielen eine entscheidende Rolle dabei, diese dynamischen Mengen **effektiv zu repräsentieren**
- Ein wichtiger Aspekt dabei ist, dass **verschiedene Datenstrukturen unterschiedliche Effizienzniveaus** in Bezug auf ihre Fähigkeit zur Durchführung von Manipulationen oder Operationen aufweisen können.
- Die Wahl der am besten geeigneten Datenstruktur für einen bestimmten Algorithmus sollte daher in erster Linie auf ihrer **Effizienz** basieren, insbesondere in Bezug auf die **spezifischen Operationen, die der Algorithmus benötigt**.

- Datenstrukturen dienen zur **Repräsentation** und **Verwaltung** von Sammlungen von **Elementen** oder **Datensätzen**.
- Jedes Element in einer Datenstruktur besteht aus einer Reihe von **Attributen**.
- Innerhalb dieser Attribute kann es spezielle Attribute geben, die als **Schlüssel** bezeichnet werden. Diese dienen zur eindeutigen Identifizierung eines Elements.

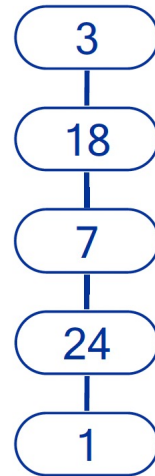
```
class Node:
    def __init__(self, key, data=None):
        self.key = key
        self.data = data
```

- **INSERT (D, x), DELETE (D, x)**
Einfügen / Löschen von Element mit Schlüssel x
- **SEARCH (D, x)**
Suche Element mit Schlüssel x
- **SIZE (D)**
Anzahl der Elemente in D
- **MAX (D), MIN (D)**
Maximum / Minimum von D liefert das Element mit dem größten / kleinsten Schlüssel
- **SUCC (D, x), PRED (D, x)**
Nachfolger / Vorgänger von Element mit Schlüssel x liefert Element mit nächstgrößerem / nächstkleineren Schlüssel

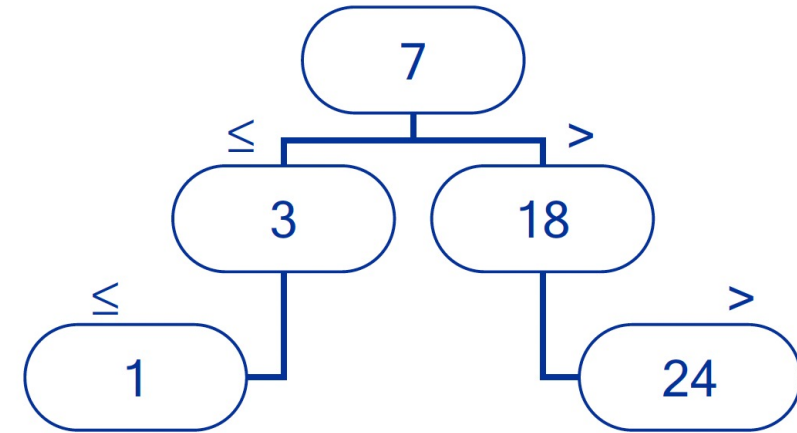
- **Feld (Array)**
 - Zugriff auf ein Element über den Index
- **Verkettete Listen (Linked Lists)**
 - Element besitzt Verweis auf das folgende Element
- **Warteschlange (Queue)**
 - Elemente können nur in gleicher Reihenfolge des Einfügens gelesen oder gelöscht werden
- **Stapel (Stack)**
 - Elemente können nur in umgekehrter Reihenfolge des Einfügens gelesen oder gelöscht werden
- **Graphen und Bäume**
 - Elemente besitzen variable Anzahl von Verweisen auf weitere Elemente

MOTIVATION: REPRÄSENTATION DER ZAHLEN (3, 18, 7, 24,1):

Repräsentation als unsortierte Liste

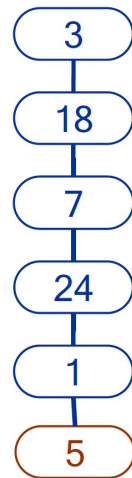


Repräsentation als Binärbaum

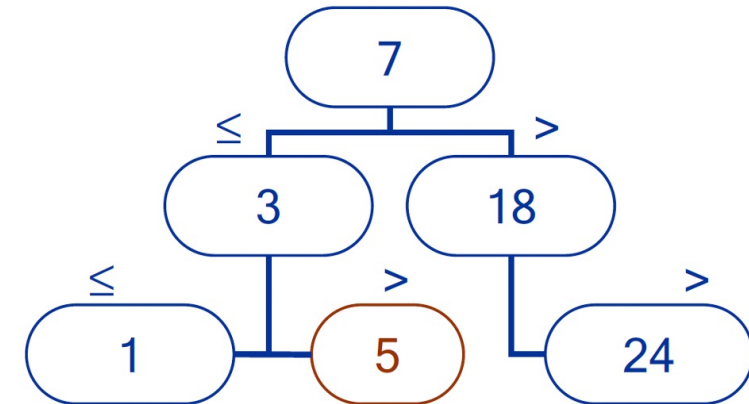


MOTIVATION: EINFÜGEN EINES ELEMENTS "5"

Repräsentation als Liste



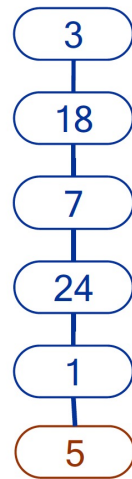
Repräsentation als Binärbaum



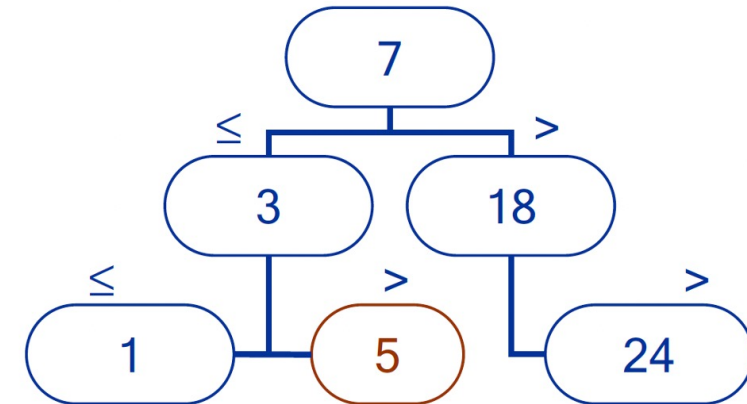
EINFACHER FÜR DIE UNSORTIERTE LISTE

MOTIVATION: SUCHEN DES ELEMENTS “5”

Repräsentation als Liste



Repräsentation als Binärbaum



EFFIZIENTER FÜR DEN BINÄRBAUM

Feld - Array

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

- **Feld hat feste Anzahl von Elementen zur Repräsentation von Elementen**

```
>>> cars = ["Ford", "Volvo", "BMW"]  
>>> cars  
['Ford', 'Volvo', 'BMW']
```

0	1	2
„Ford“	„Volvo“	„BMW“

Hinweis:

- Erweiterung des Arrays kann in Python einfach durch den Append-Befehl erreicht werden, da Python immer direkt mit **Listen** arbeitet
- In **anderen Programmiersprachen** ist die Verlängerung eines Arrays jedoch teilweise **deutlich aufwändiger**

- **Einfügen eines Elements an bestimmter Position in Python**

```
def insert_element(array, index, element):  
    return array[:index] + [element] + array[index:]
```

Alternativ: Verwendung der insert()-Methode von Listen

- **Löschen eines Elements an bestimmter Position in Python:**

```
def delete_element(array, index):  
    return array[:index] + array[index + 1:]
```

Alternativ: Verwendung der pop() oder del-Methode von Listen

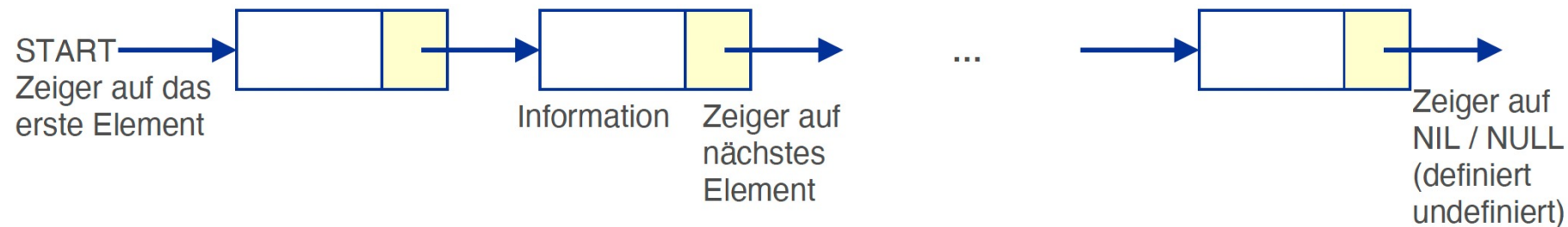
- **Suchen:** Komplexität abhängig davon, ob Daten sortiert sind oder nicht

Verkettete Liste

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

- Dynamische Datenstruktur
- Zahl der Elemente während der Laufzeit frei wählbar
- Elemente bestehen aus einfachen oder zusammengesetzten Datentypen
- Elemente sind durch Zeiger / Referenzen auf ein folgendes Element verbunden
- Typen: einfache oder doppelte Verkettung

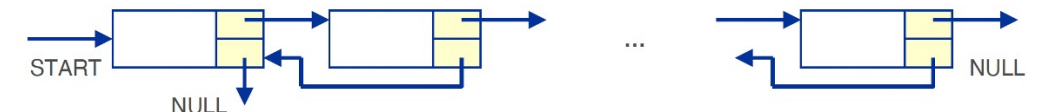


- geringer Mehrbedarf an Speicher
- Einfügen und Löschen von Elementen erfolgt ohne Umkopieren anderer Elemente
- Zahl der Elemente kann beliebig verändert werden
- kein direkter Zugriff auf Elemente (Liste muss durchlaufen werden)

Einfach verkettete Liste



Doppelt verkettete Liste



VERKETTETE LISTE – INSERT UND PRINT

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        if not self.head:
            self.head = Node(data)
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = Node(data)

    def print_list(self):
        current = self.head
        while current:
            print(current.data)
            current = current.next
```

```
# Nutzung der verlinkten Liste
linked_list = LinkedList()
linked_list.insert('A')
linked_list.insert('B')
linked_list.insert('C')

linked_list.print_list()
```



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

[...]
```

```
def delete(self, data):
    if self.head is None:
        return
    if self.head.data == data:
        self.head = self.head.next
        return
    current = self.head
    while current.next is not None:
        if current.next.data == data:
            current.next = current.next.next
            return
        current = current.next
```

```
# Nutzung der verlinkten Liste
linked_list = LinkedList()
linked_list.insert('A')
linked_list.insert('B')
linked_list.insert('C')

linked_list.print_list()

linked_list.delete('B')

linked_list.print_list()
```

- Bitte erweitern Sie die Implementierung der einfach verketteten Liste auf eine doppelt verkettete Liste.
- Bitte implementieren Sie die Funktion `get(i)`, welche Ihnen das *i*-te Element der verketteten Liste zurückgibt
- Ergänzen Sie die Funktion „`insertAfterElement`“, die ein neues Element nach einem schon in der Liste enthaltenem Element hinzufügt

- **Dynamische Speicherverwaltung:** Verlinkte Listen sind besonders nützlich, wenn die Menge an Daten, die gespeichert werden muss, im Voraus nicht bekannt ist. Sie ermöglichen die dynamische Speicherallokation.
- **Implementierung von fortgeschrittenen Datenstrukturen:** Verlinkte Listen dienen oft als Bausteine für komplexere Datenstrukturen wie Stacks, Queues, Hash-Tabellen und sogar Graphen.
- **Verwaltung von Listen in Datenbanken:** Verlinkte Listen können zur Implementierung von Listen in Datenbanksystemen verwendet werden, insbesondere wenn Elemente häufig hinzugefügt oder entfernt werden.
- **Implementierung von Navigation in Softwareanwendungen:** Beispielsweise können doppelt verlinkte Listen verwendet werden, um vorwärts und rückwärts durch eine Reihe von Bildern oder Seiten zu navigieren.
- **Speicher-effiziente Verwendung bei großen Datensätzen:** Da verlinkte Listen nur Speicherplatz für die Elemente beanspruchen, die tatsächlich in der Liste sind, sind sie eine effiziente Wahl, wenn mit großen Datenmengen gearbeitet wird, die sich häufig ändern.

Warteschlange - Queue

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

- Dynamische Menge mit speziellen Zugriffsoperationen
- **Funktioniert nach dem FIFO-Prinzip – First In / First Out**
 - Das am längsten wartende Element wird als nächstes aus der Liste entfernt
- Realisierung als verkettete Liste oder Array
- **Anfang und Ende der Schlange sind immer bekannt**



IMPLEMENTIERUNG: WARTESCHLANGE - QUEUE

```
class Queue:
    def __init__(self):
        self.queue = []

    # Add an element
    def enqueue(self, item):
        self.queue.append(item)

    # Remove an element
    def dequeue(self):
        if len(self.queue) < 1:
            return None
        return self.queue.pop(0)

    # Display the queue
    def display(self):
        print(self.queue)
```

```
# Anwendung der Queue
q = Queue()
q.enqueue("A")
q.enqueue("B")
q.enqueue("C")

q.display()

q.dequeue()

q.display()
```

- **Datenpuffer:** Queues werden oft verwendet, um Daten zwischen zwei Prozessen zu puffern. Zum Beispiel kann ein Prozess Daten in eine Queue schreiben, während ein anderer Prozess Daten liest.
- **Druckaufträge:** Wenn mehrere Druckaufträge gleichzeitig an einen Drucker gesendet werden, wird eine Queue verwendet, um diese Aufträge zu verwalten.
- **CPU-Scheduling:** In Betriebssystemen werden Queues verwendet, um zu bestimmen, welcher Prozess als nächstes die CPU erhält. Die Prozesse warten in einer Queue, bis sie an der Reihe sind.
- **Webserver-Anfragen:** Webserver verwenden Queues, um eingehende Anfragen zu verwalten. Wenn mehrere Anfragen gleichzeitig eingehen, werden sie in eine Queue eingefügt und dann in der Reihenfolge ihres Eintreffens bearbeitet.
- **Message Queuing Service:** In verteilten Systemen werden Queues verwendet, um Nachrichten zwischen den Komponenten zu übertragen. Jede Nachricht wird in eine Queue eingefügt und dann vom Empfänger abgerufen.
- **Tastatureingaben:** Wenn Sie auf Ihrer Tastatur tippen, werden die Tastenanschläge in einer Queue gespeichert und dann von Ihrem Betriebssystem verarbeitet.

Stapel – Kellerspeicher - Stacks

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

- Dynamische Menge mit speziellen Zugriffsoperationen
- **Funktioniert nach dem LIFO-Prinzip – Last In / First Out**
 - Das am kürzesten wartende Element wird als nächstes aus der Liste entfernt
- Realisierung als verkettete Liste oder Array
- **Nur das oberste Element des Stapel ist bekannt**



```
class Stack:
    def __init__(self):
        self.stack = []

    # Add an element
    def push(self, item):
        self.stack.append(item)

    # Remove an element
    def pop(self):
        if len(self.stack) < 1:
            return None
        removed_item = self.stack[-1]
        del self.stack[-1]
        return removed_item

    # Display the stack
    def display(self):
        print(self.stack)
```

```
# Anwendung des Stacks
s = Stack()
s.push("A")
s.push("B")
s.push("C")

s.display()

s.pop()

s.display()
```

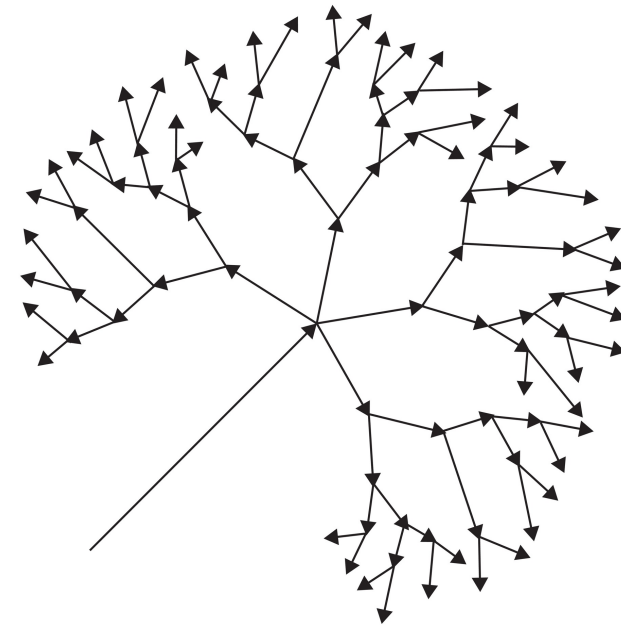
- **Rückgängig / Wiederholen Funktionen:** In vielen Softwareanwendungen wie Texteditoren oder Grafikprogrammen wird ein Stack verwendet, um die zuletzt durchgeführten Aktionen zu speichern. Wenn der Benutzer die "Rückgängig"-Funktion ausführt, wird die oberste Aktion vom Stack entfernt und rückgängig gemacht.
- **Aufrufstack in Programmiersprachen:** Wenn Funktionen oder Methoden in einer Programmiersprache aufgerufen werden, werden die Details dieser Aufrufe, einschließlich der zurückzukehrenden Adressen und der lokalen Variablen, auf einem Stack gespeichert.
- **Kontrolle von Programmausführungsflüssen:** Stacks werden verwendet, um die Kontrollflüsse in Programmiersprachen zu verwalten, beispielsweise in geschachtelten Schleifen und bedingten Anweisungen.
- **Syntaxprüfung in Compilern:** Stacks helfen beim Überprüfen der Syntax von Ausdrücken in Compilern. Sie prüfen, ob Klammern, Anführungszeichen oder HTML/XML-Tags korrekt gepaart sind.
- **Speicherverwaltung in Betriebssystemen:** In Betriebssystemen werden Stacks verwendet, um Speicher effizient zu verwalten. Jeder Prozess bekommt einen eigenen Stack, der bei Bedarf erweitert oder verkleinert werden kann.

Binäre Suchbäume

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

- Ein Baum in der Informatik ist eine Menge von Knoten und Kanten, die eine spezielle hierarchische Struktur aufweisen.
- Ein Baum besitzt immer einen speziellen Knoten, bekannt als die Wurzel (root), außer im Fall eines leeren Baums, der keine Knoten hat.
- Jeder Knoten im Baum (mit Ausnahme der Wurzel) ist durch genau eine Kante mit einem anderen Knoten verbunden, der als sein Elternknoten (parent) bezeichnet wird. Dieser Knoten wird das Kind (child) des Elternknotens genannt.
- Ein Knoten, der keine Kinder hat, wird als Blatt (leaf) bezeichnet. Knoten, die mindestens ein Kind haben, werden als innere Knoten bezeichnet.
- Jeder Knoten in einem Baum hat höchstens einen Elternknoten, aber kann mehrere Kindknoten haben.
- Ein spezieller Typ von Baum ist der Binärbaum (binary tree), bei dem jeder Knoten höchstens zwei Kinder hat. Allgemeiner können Bäume existieren, in denen jeder Knoten bis zu n Kinder haben kann; solche Bäume werden als n -äre Bäume (n-ary trees) bezeichnet.



Ein Binärbaum mit **Suchbaum-Eigenschaft**, auch bekannt als Binärer Suchbaum (binary search tree, BST), ist eine spezielle Art von binärem Baum, der folgende Eigenschaften aufweist:

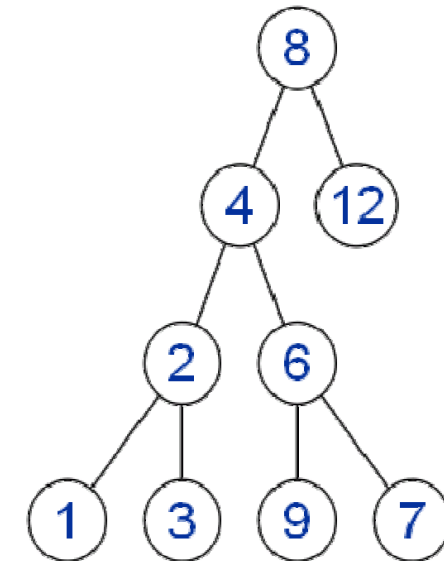
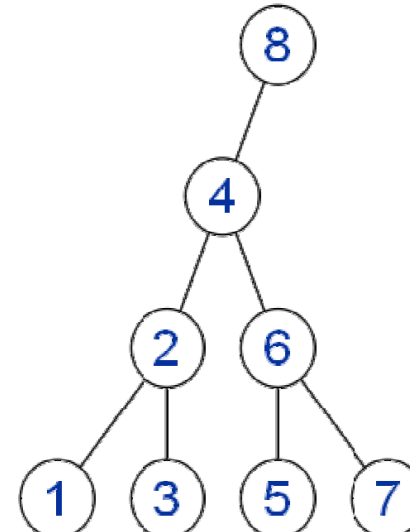
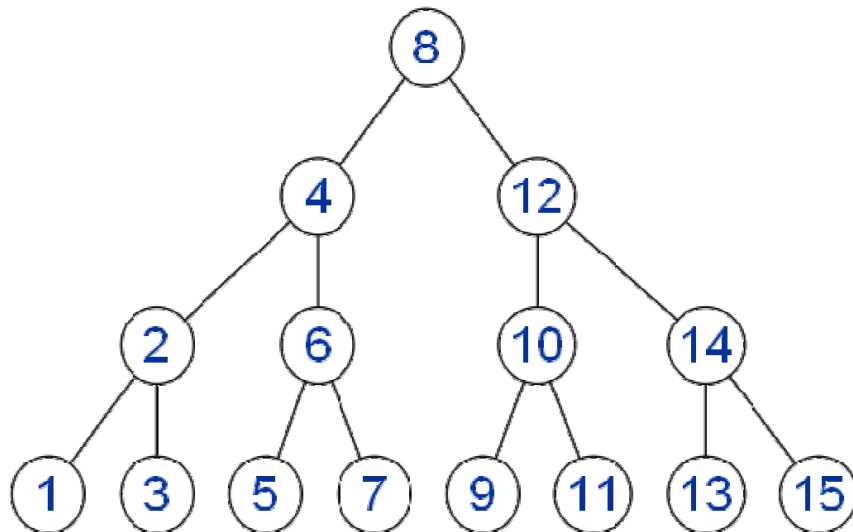
- Jeder Knoten im Baum besitzt **höchstens zwei Kinder**, das linke Kind und das rechte Kind.
- Für jeden Knoten im Baum gilt, dass **alle Werte im linken Teilbaum kleiner als der Wert des Knotens sind** und alle Werte im **rechten Teilbaum größer als der Wert des Knotens sind**.

Diese Eigenschaften gelten für jeden Knoten im Baum, nicht nur für die Wurzel.

Die **Suchbaum-Eigenschaft** macht binäre Suchbäume besonders nützlich für Such- und Sortieroperationen, da sie es ermöglichen, Daten effizient zu durchsuchen. Eine in-order Traversierung eines binären Suchbaums liefert beispielsweise eine sortierte Liste der in ihm enthaltenen Elemente.

Es ist zu beachten, dass **binäre Suchbäume ausgeglichen sein sollten**, um eine optimale Leistung zu gewährleisten. Ein unausgewogener Baum kann im schlimmsten Fall zu Operationen führen, die linear mit der Anzahl der Knoten im Baum skaliert, anstatt logarithmisch.

WELCHER BAUM IST KEIN BINÄRER SUCHBAUM?



```
class Node:
    def __init__(self, data):
        self.left = None
        self.right = None
        self.data = data

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert(data, self.root)

    def _insert(self, data, node):
        if data < node.data:
            if node.left is None:
                node.left = Node(data)
            else:
                self._insert(data, node.left)
        else:
            if node.right is None:
                node.right = Node(data)
            else:
                self._insert(data, node.right)
```

```
def print_tree(self):
    if self.root is not None:
        self._print_tree(self.root)

def _print_tree(self, node):
    if node is not None:
        self._print_tree(node.left)
        print(str(node.data))
        self._print_tree(node.right)

# Verwendung des binären Baums
bt = BinaryTree()
bt.insert(10)
bt.insert(5)
bt.insert(15)
bt.insert(3)

bt.print_tree()
```


IMPLEMENTIERUNG: BINÄRER SUCHBAUM - DELETE

```
def delete(self, data):
    self.root = self._delete(self.root, data)

def _delete(self, node, data):
    if node is None:
        return node
    if data < node.data:
        node.left = self._delete(node.left, data)
    elif data > node.data:
        node.right = self._delete(node.right, data)
    else:
        if node.left is None:
            temp = node.right
            node = None
            return temp
        elif node.right is None:
            temp = node.left
            node = None
            return temp
        temp = self._find_min(node.right)
        node.data = temp.data
        node.right = self._delete(node.right, temp.data)
    return node
```

```
def _find_min(self, node):
    if node.left is None:
        return node
    else:
        return self._find_min(node.left)
```

- **Suchalgorithmen:** Aufgrund ihrer Eigenschaften sind BSTs hervorragend geeignet für Suchoperationen. Sie ermöglichen schnelle Suchen, Einfügungen und Löschungen von Daten und werden daher oft in Suchalgorithmen eingesetzt.
- **Dynamisches Sortieren:** BSTs können zum dynamischen Sortieren von Daten verwendet werden. Im Gegensatz zu statischen Sortieralgorithmen, die eine einmalige Sortierung durchführen, ermöglichen BSTs das fortlaufende Einfügen und Entfernen von Daten, während die sortierte Reihenfolge erhalten bleibt.
- **Datenbanken:** Viele Datenbanksysteme verwenden BSTs oder Variationen davon (wie B-Bäume und T-Bäume), um Daten effizient zu indizieren und abzurufen.
- **Dateisysteme:** Ähnlich wie bei Datenbanken werden BSTs auch in Dateisystemen verwendet, um Dateien schnell zu finden und zu sortieren.
- **Speicherbelegung:** In Betriebssystemen können BSTs verwendet werden, um Speicherblöcke effizient zu verwalten, indem sie Speicheradressen speichern und organisieren.
- **Grafische Datenverarbeitung:** In der Computergrafik können BSTs zur Speicherung von Szeneninformationen verwendet werden, z.B. in Form von BSP-Bäumen (Binary Space Partitioning Trees), um schnelle Rendering-Operationen zu ermöglichen.